O D I N

Trading Bot / ODIN Assistant

مستند معماری فنی

وضعیت: تکمیل شده و آماده ورود به دوره سایه

تاریخ مستند: ۲۰۲۶-۰۹-۲۴

هدف سند: ثبت دقیق معماری فعلی، مرز مسئولیت‌ها، قراردادها، Pluginها، Strategy Layer، Pipeline، ارتباط Python و Android و اصول توسعه آینده.

فهرست مطالب

۱. معرفی معماری 4
۲. تصویر کلی معماری 5
۳. اصل بنیادی: Core نباید Feature را بشناسد 6
۴. لایه‌های اصلی پروژه 7
۵. Core 8
۶. Contractهای اصلی فعلی 9
۷. Plugin Manifest 10
۸. Plugin Registry 11
۹. Dependency Resolver 12
۱۰. Lifecycle 13
۱۱. Failure Isolation 14
۱۲. Event Bus 15
۱۳. Pipeline 16
۱۴. Judge Architecture 19
۱۵. Veto System 20
۱۶. Evidence System 21
۱۷. Risk Calculation 22
۱۸. Strategy Layer 23
۱۹. Strategy Contract فعلی 24
۲۰. Strategyهای فعال فعلی 26
۲۱. Strategy Agreement Gate 29
۲۲. Fail-Closed Strategy Behavior 30
۲۳. Config Architecture 31
۲۴. Python Runtime 32
۲۵. Android Runtime 33
۲۶. Python ↔ JavaScript Parity 34
۲۷. Strategy Golden Tests 35
۲۸. Journal 37
۲۹. Notification 38
۳۰. Report 39
۳۱. Alerts 40
۳۲. Data Layer 41
۳۳. Fundamental Layer 42
۳۴. Session Layer 43
۳۵. Runtime Flow کامل 44
۳۶. Scheduler 46
۳۷. Native Boundary 47

۳۸. استقلال Windows 48
۳۹. قوانین توسعه Plugin جدید 49
۴۰. قوانین توسعه آینده 50
۴۱. مسیر معماری آینده برای Auto Trading 51
۴۲. چرا این معماری برای ODIN مناسب است؟ 52
۴۳. وضعیت فعلی معماری 53
۴۴. ساختار مفهومی Repository 54
۴۵. تعریف نهایی معماری 56
۴۶. مرزهای غیرقابل شکستن 57

۱. معرفی معماری

ODIN یک Modular Monolith با معماری Contract-Based و Plugin-Driven است.

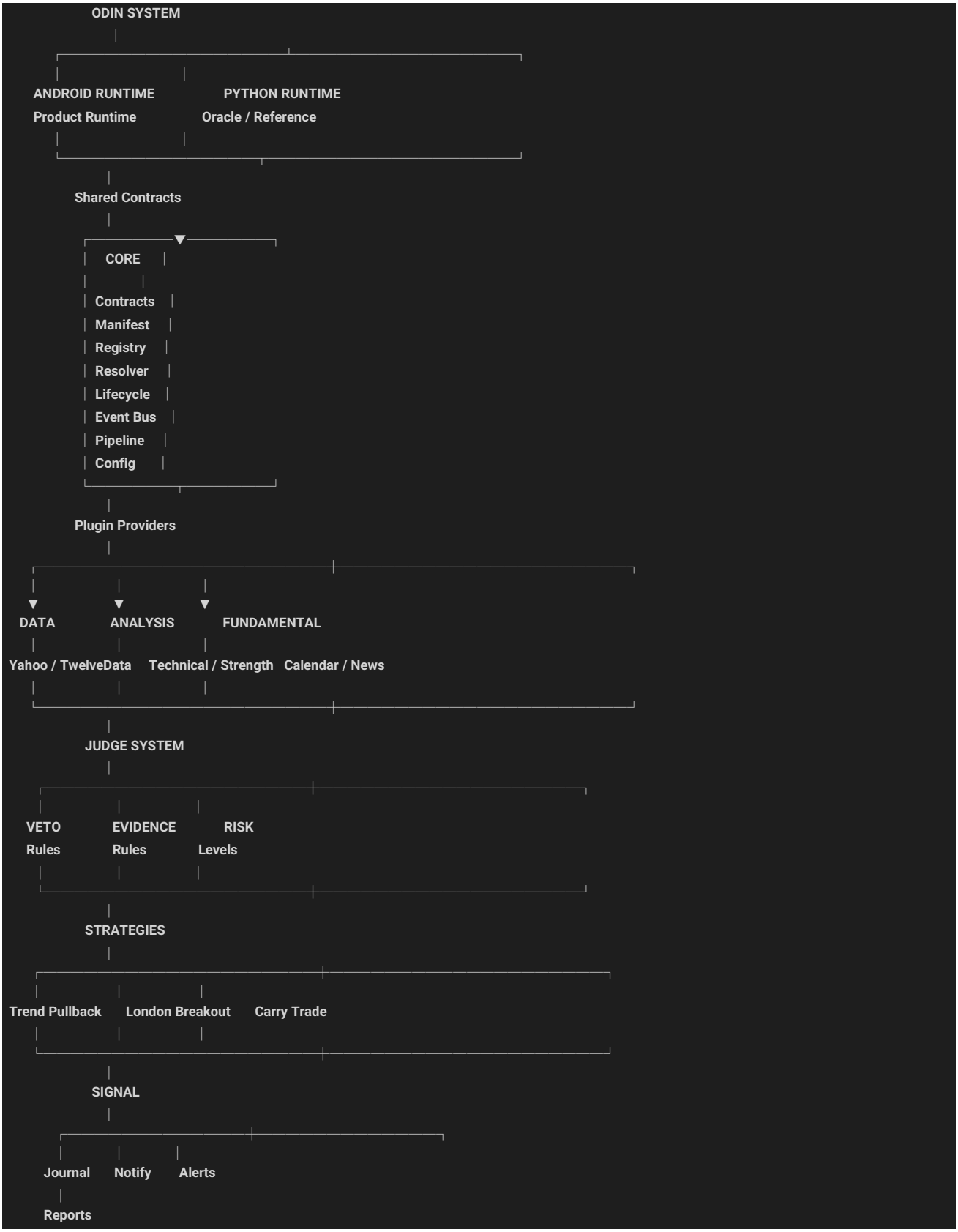
هدف اصلی معماری این است که Core پروژه کوچک، پایدار و مستقل از قابلیت‌های معاملاتی باقی بماند و قابلیت‌های مختلف به‌صورت Pluginهای مستقل در اطراف آن قرار بگیرند.

معماری فعلی بر پنج اصل اصلی استوار است:

- **Core کوچک و مستقل**
- **Contract-Based Integration**
- **Plugin-Driven Features**
- **Pipeline و Lifecycle قطعی**
- **Behavior Preservation و Python/JS Parity**

در این معماری Core نباید بداند که یک قابلیت دقیقاً با Strategy، TradingView، News، RSI، Yahoo یا Telegram چگونه کار می‌کند. Core فقط Contract و وضعیت Pluginها را می‌شناسد.

۲. تصویر کلی معماری



۳. اصل بنیادی: Core نباید Feature را بشناسد

Core فقط شکل و قرارداد قابلیت‌ها را می‌شناسد.

برای مثال Core می‌داند:

```
odin.data.market@1
odin.analysis.technical@1
odin.judge.engine@1
odin.strategy@1
```

اما Core نباید بداند:

```
Yahoo چیست؟
RSI چگونه محاسبه می‌شود؟
Trend Pullback چگونه کار می‌کند؟
Telegram چگونه ارسال می‌شود؟
TradingView چه APIای دارد؟
```

این جزئیات متعلق به Plugin یا Feature Layer هستند.

نتیجه:

```
Core = قرارداد + مدیریت Plugin + اجرای Pipeline

Plugin = منطق واقعی قابلیت
```

۴. لایه‌های اصلی پروژه

معماری پروژه را می‌توان به این لایه‌ها تقسیم کرد:

```
Application / Runtime
        │
        ▼
Pipeline / Orchestration
        │
        ▼
Core
        │
        ├──── Contracts
        ├──── Registry
        ├──── Resolver
        ├──── Lifecycle
        ├──── Event Bus
        ├──── Config Bridge
        └──── Pipeline Runner
        │
        ▼
Plugins
        │
        ├──── Data
        ├──── Analysis
        ├──── Fundamental
        ├──── Judge
        ├──── Strategy
        ├──── Journal
        ├──── Notify
        ├──── Alerts
        └──── Report
        │
        ▼
Existing Feature Modules
```

۵. Core

مسیر اصلی:

```
src/core/
```

و در Android:

```
android/app/src/main/assets/www/js/core.js
```

Core شامل اجزای زیر است.

۵٫۱ Contracts

فایل:

```
src/core/contracts.py
```

Contractها interfaceهای رسمی سیستم هستند.

در Python از `typing.Protocol` و `runtime_checkable@` استفاده شده است.

Contractها با شناسه نسخه‌دار تعریف می‌شوند:

```
odin.<domain>.<capability>@<major>
```

مثال:

```
odin.data.market@1
odin.analysis.technical@1
odin.judge.engine@1
odin.strategy@1
```

قانون Versioning

تغییر سازگار:

```
odin.strategy@1
```

باقی می‌ماند.

تغییر ناسازگار:

```
odin.strategy@2
```

و در صورت نیاز نسخه قبلی تا پایان Migration نگهداری می‌شود.

۶. Contractهای اصلی فعلی

مسئولیت	Contract
دریافت Market Data	odin.data.market@۱
TradingView Confirmation	odin.data.tv@۱
Technical Analysis	odin.analysis.technical@۱
Currency Strength	odin.analysis.strength@۱
وضعیت Session	odin.session@۱
Economic Calendar	odin.fundamental.calendar@۱
News	odin.fundamental.news@۱
Judge اصلی	odin.judge.engine@۱
Veto Rule	odin.judge.veto@۱
Evidence Rule	odin.judge.evidence@۱
Risk/SL/TP Calculation	odin.judge.risk@۱
Strategy Evaluation	odin.strategy@۱
Journal	odin.journal@۱
Notification	odin.notify@۱
Price Alert	odin.alerts.price@۱
Report Rendering	odin.report@۱

۷. Plugin Manifest

هر Plugin یک Manifest دارد.

Manifest مشخص می‌کند:

```
Plugin ID
Version
Provides
Requires
Config Binding
Platforms
Pipeline Stage
Priority
Optional
Factory
```

نمونه مفهومی:

```
{
  "id": "strategy-trend-pullback",
  "version": "1.0.0",
  "provides": ["odin.strategy@1"],
  "requires": [],
  "config": {
    "section": "strategies",
    "enabled_key": "trend_pullback.enabled",
    "default": true
  },
  "platforms": ["desktop", "android"],
  "stage": "judge",
  "priority": 10,
  "optional": true
}
```

Manifest داده است، نه منطق Business.

۸. Plugin Registry

مسیر:

```
src/core/registry.py
```

Registry مرکز ثبت و Discovery Pluginها است.

وظایف:

- **ثبت Plugin**
- **جلوگیری از Duplicate ID**
- **جست‌وجوی Plugin با ID**
- **پیدا کردن Provider یک Contract**
- **مرتب‌سازی Providerها**
- **Enable / Disable**
- **نگهداری وضعیت Plugin**
- **ارائه وضعیت Pluginها به UI**

چند Plugin می‌توانند یک Contract را ارائه کنند.

مثلاً:

```
odin.data.market@1
        │
        ├──── data-yahoo
        ├──── data-twelvedata
        └──── data-auto
```

انتخاب Provider بر اساس منطق و Config انجام می‌شود و تصادفی نیست.

۹. Dependency Resolver

مسیر:

```
src/core/resolver.py
```

Resolver وابستگی‌های Pluginها را بررسی و ترتیب اجرای صحیح را مشخص می‌کند.

Dependencyها از طریق:

```
requires
```

در Manifest تعریف می‌شوند.

مثال:

```
fundamental-calendar
        |
        └──── requires → odin.session@1
```

رفتار وابستگی مفقود

اگر Dependency موجود نباشد:

```
Plugin → disabled / unavailable
```

و سیستم Crash نمی‌کند.

رفتار Circular Dependency

حلقه Dependency یک خطای معماری است و باید با خطای واضح گزارش شود.

مثال:

```
A → B → C → A
```

این وضعیت Fail-Loud است.

۱۰. Lifecycle

مسیر:

```
src/core/lifecycle.py
```

Pluginها چرخه حیات مشخص دارند.

```
REGISTERED
   ↓
RESOLVED
   ↓
INITIALIZED
   ↓
STARTED
   ↓
STOPPED
   ↓
DISPOSED
```

در صورت خطای Plugin، سیستم آن را قرنطینه می‌کند و وضعیت خطا را نگه می‌دارد.

هدف:

```
Plugin Failure ≠ Application Crash
```

۱۱. Failure Isolation

یکی از اصول مهم ODIN:

> *خطای یک قابلیت نباید باعث Crash کل سیستم شود.*

رفتار استاندارد:

```
Plugin Error
    ↓
Capture
    ↓
Plugin marked unavailable / failed
    ↓
Error logged
    ↓
Pipeline continues where possible
```

و مهم‌تر:

```
Unavailable ≠ Fake Data
```

اگر داده موجود نیست، سیستم نباید حدس بزند.

۱۲. Event Bus

مسیر:

```
src/core/bus.py
```

Event Bus یک Pub/Sub داخلی و synchronous است.

ویژگی‌ها:

- **بدون Dependency خارجی**
- **ترتیب اجرای قطعی**
- **Listenerهای مستقل**
- **Isolation خطای Listener**
- **بدون Threading داخلی**
- **عدم تغییر نتیجه Pipeline توسط Listener**

رویدادهای مهم:

```
cycle.start
market.collected
journal.resolved
fundamental.collected
vetoes.computed
judge.done
signal.created
signal.sent
alerts.fired
report.rendered
telegram.sent
cycle.end
plugin.registered
plugin.failed
plugin.disabled
stage.start
stage.done
```

Event Bus برای Side Effectهای کنترل‌شده و Observability استفاده می‌شود، نه برای تغییر پنهانی منطق اصلی تصمیم‌گیری.

۱۳. Pipeline

مسیر:

```
src/core/pipeline.py
```

Pipeline چرخه تحلیل را به مراحل مشخص و نام‌دار تقسیم می‌کند.

ترتیب فعلی:

```
1. collect_market
2. journal_pre
3. collect_fundamental
4. compute_vetoes
5. judge
6. render
7. dispatch_signals
8. price_alerts
9. chart_cache
10. dashboard
11. archive_notify
```

۱۳٫۱ collect_market

مسئول:

- **Market Data**
- **Technical Analysis**
- **Currency Strength**
- **TradingView**

خروجی اصلی:

```
datasets
analyses
ranking
tv_map
```

۱۳٫۲ journal_pre

قبل از ادامه تحلیل:

- **سیگنال‌های باز بررسی می‌شوند.**
- **نتایج قابل resolve مشخص می‌شوند.**
- **آمار Journal به‌روزرسانی می‌شود.**

این ترتیب عمدی است.

حتی اگر Market Data در دسترس نباشد، وضعیت Signalهای باز نباید بی‌دلیل متوقف شود.

۱۳٫۳ collect_fundamental

مسئول:

```
Economic Calendar
News
```

۱۳٫۴ compute_vetoes

Vetoهای نمادمحور محاسبه می‌شوند.

۱۳٫۵ judge

مرکز تصمیم‌گیری تحلیلی ODIN است.

Judge موارد زیر را ترکیب می‌کند:

```
Vetoes
Evidence
Score
Strategy Agreement
Risk Levels
```

۱۳٫۶ render

تولید:

- Console Report
- Signal Report
- Judge Summary
- سایر Reportها

۱۳٫۷ dispatch_signals

مسئول:

- ضداسپم
- ارسال Signal
- Journal کردن Signal

۱۳٫۸ price_alerts

هشدارهای قیمتی فعال را بررسی می‌کند.

chart_cache ۱۳٫۹

داده لازم برای نمودار را Cache می‌کند.

dashboard ۱۳٫۱۰

داده ساختاریافته Dashboard را آماده می‌کند.

archive_notify ۱۳٫۱۱

گزارش نهایی را Archive و در صورت نیاز ارسال می‌کند.

۱۴. Judge Architecture

Judge هنوز مرکز تصمیم نهایی تحلیلی است.

ساختار:



۱۵. Veto System

Vetoها Rule Plugin هستند.

Vetoهای فعلی:

```
veto-data
veto-weekend
veto-tf-conflict
veto-range
veto-event
veto-vol-spike
veto-breaking-news
```

Veto یعنی:

```
این شرایط اجازه ادامه تصمیم‌گیری را نمی‌دهد.
```

Veto نباید توسط Strategy دور زده شود.

۱۶. Evidence System

Evidenceها نیز Plugin هستند.

Evidenceهای فعلی:

```
ev-trend
ev-level
ev-fundamental
ev-momentum
ev-strength
ev-news
ev-tv
ev-session
```

Evidenceها به Signal امتیاز می‌دهند.

سیستم فعلی Judge حداکثر امتیاز تعریف‌شده خود را دارد و آستانه از Config می‌آید.

۱۷. Risk Calculation

Risk Calculation در Strategy قرار ندارد.

محاسبه مواردی مانند:

```
Entry
SL
TP
Reward/Risk
SL Cap
```

در لایه Judge/Risk انجام می‌شود.

این جداسازی برای آینده مهم است:

```
Strategy
≠
Risk
≠
Execution
```

۱۸. Strategy Layer

Strategy Layer یکی از لایه‌های جدید معماری است.

Contract:

```
odin.strategy@1
```

مسیر Python:

```
src/strategies/
src/plugins/strategies.py
```

مسیر Android:

```
android/app/src/main/assets/www/js/strategies.js
```

Strategyها توسط Registry به‌صورت Plugin ثبت می‌شوند.

۱۹. Strategy Contract فعلی

خروجی Strategy در معماری فعلی:

```
key
name_fa
direction
strength
proposes
reasons_fa
detail_fa
```

مثال:

```
{
  "key": "trend_pullback",
                                                    "name_fa": "روند + پولبک",
  "direction": "BUY",
  "strength": 0.78,
  "proposes": true,
  "reasons_fa": [
                                                      "روند صعودی...",
                                                      "پولبک شناسایی شد...",
                                                      "تأیید ادامه حرکت..."
  ],
                                                "detail_fa": "روند + پولبک: پیشنهاد BUY"
}
```

معنی فیلدها

key

شناسه ثابت Strategy.

name_fa

نام قابل نمایش.

direction

فقط:

```
BUY
SELL
NONE
```

strength

قدرت Setup در بازه:

0..1

proposes

آیا Strategy خودش پیشنهاددهنده جهت معامله است یا فقط در توافق شرکت می‌کند.

reasons_fa

دلایل قابل نمایش.

detail_fa

خلاصه قابل نمایش.

۲۰. Strategyهای فعال فعلی

سه Strategy در پروژه وجود دارند:

```
strategy-trend-pullback
strategy-london-breakout
strategy-carry
```

۲۰٫۱ Trend Pullback

شناسه:

```
trend_pullback
```

هدف:

شناسایی روند و پیدا کردن Pullback در جهت روند.

عوامل اصلی:

- Trend
- H۴/H۱ Alignment
- ADX
- RSI Pullback Zone
- RSI Reversal
- تأیید کندل H۱

خروجی می‌تواند:

```
BUY
SELL
NONE
```

باشد.

این Strategy یک direction proposer است.

۲۰٫۲ London Breakout

شناسه:

```
london_breakout
```

هدف:

شناسایی شکست Range مرتبط با Session لندن.

عوامل اصلی:

- **زمان Session**
- **Range**
- **حداقل/حداکثر عرض Range**
- **ATR**
- **Breakout Margin**
- **جهت شکست**

در صورت Breakout معتبر:

```
BUY / SELL
```

در غیر این صورت:

```
NONE
```

این Strategy نیز direction proposer است.

Carry Trade ۲۰٫۳

شناسه:

```
carry
```

هدف:

استفاده از اختلاف نرخ بهره و هم‌جهتی Macro/Fundamental برای ایجاد یک سوگیری کندتر.

عوامل اصلی:

- **اختلاف نرخ سیاستی**
- **Monetary Policy Bias**
- **Currency Strength**
- **News/Fundamental Alignment**
- **نرخ‌های Config شده و `as_of`**

Carry در معماری فعلی:

```
proposes = false
```

است.

یعنی Carry به‌تنهایی منبع جهت Signal نیست و در Strategy Agreement به‌عنوان پشتیبان/سوگیری Fundamental شرکت می‌کند.

اگر داده نرخ موجود نباشد:

```
NONE
```

و اگر اختلاف نرخ کافی نباشد نیز:

```
NONE
```

تولید می‌شود.

۲۱. Strategy Agreement Gate

Strategyها مستقیماً Signal نهایی تولید نمی‌کنند.

جریان:

```
Technical Setup
    ↓
Judge
    ↓
Strategy Evaluation
    ↓
Agreement Gate
    ↓
Final Signal
```

اگر Judge مثلاً به:

```
BUY
```

رسیده باشد، Strategyها بررسی می‌شوند.

Strategyهایی که:

```
direction == BUY
```

دارند، موافق محسوب می‌شوند.

برای Strategyهای proposer، حداقل یک پیشنهاددهنده باید وجود داشته باشد.

اگر توافق کافی نباشد:

```
NO_STRATEGY
```

ثبت می‌شود و Signal صادر نمی‌شود.

۲۲. Fail-Closed Strategy Behavior

اگر همه Strategyها غیرفعال باشند و Strategy Gate فعال باشد:

```
No active strategy
      ↓
NO_STRATEGY
      ↓
No Signal
```

این رفتار عمداً Fail-Closed است.

هدف:

> *سیستم بدون پشتوانه Strategy، سیگنال قابل اقدام تولید نکند.*

۲۳. Config Architecture

Config موجود پروژه حفظ شده و Pluginها به Keyهای موجود متصل می‌شوند.

برای Strategyها:

```
strategies
 ├─── trend_pullback
 ├─── london_breakout
 ├─── carry
 └─── min_agree
```

فعال/غیرفعال کردن:

```
strategies.trend_pullback.enabled
strategies.london_breakout.enabled
strategies.carry.enabled
```

Override صریح Plugin نیز در معماری Registry پشتیبانی می‌شود.

۲۴. Python Runtime

Python نقش:

Oracle / Reference Implementation

را دارد.

اجزای اصلی:

```
src/core/
src/plugins/
src/strategies/
src/data/
src/analysis/
src/fundamental/
src/judge/
src/journal/
src/notify/
src/report/
```

Python مرجع رفتار محاسباتی و تست‌های طلایی است.

۲۵. Android Runtime

Android پلتفرم فعال محصول است.

معماری Android بر:

```
WebView
+
Vanilla JavaScript
+
Native Java Bridge
+
OdinService
```

استوار است.

Core JS و Plugin Registry در WebView اجرا می‌شوند.

منطق Feature نباید به Java/Kotlin منتقل شود مگر در مرزهای Native مورد نیاز.

۲۶. Python ↔ JavaScript Parity

ODIN دو Runtime دارد:

```
Python
Android JS
```

هدف:

```
Same Input
    ↓
Python Result
    =
JS Result
```

در بخش‌های حساس از Golden Fixtures استفاده می‌شود.

برای Strategyها:

```
Python Strategy
      ↕
Golden Fixture
      ↕
JavaScript Strategy
```

این موضوع مخصوصاً برای:

- **اعداد**
- **زمان**
- **NaN**
- **rounding**
- **فارسی**
- **ZWNJ**
- **ترتیب Keyها**
- **شرایط مرزی**

مهم است.

۲۷. Strategy Golden Tests

Strategy Layer دارای Golden Fixture مستقل است.

مسیر:

```
tests/js/golden/strategies_golden.json
```

تست Python:

```
tests/test_strategies.py
```

تست JS:

```
tests/js/test_strategies_switch.js
```

سناریوهای Strategy شامل موارد زیر هستند:

Trend Pullback

- Trend نامعتبر
- عدم توافق H۱/H۴
- ADX ناکافی
- RSI خارج از Zone
- RSI بدون Reversal
- Candle تأییدکننده نامعتبر
- BUY
- SELL
- Config variations

London Breakout

- خارج بودن از Window
- Weekend
- نبود Range
- Range خیلی کوچک
- Range خیلی بزرگ
- Range روز قبل
- Breakout معتبر
- Breakout لبه‌ای

- Breakout نامعتبر
- ATR ناموجود
- Config variations

Carry

- BUY
- SELL
- اختلاف نرخ ناکافی
- نرخ ناموجود
- XAU
- ارز ناشناخته
- Policy Bias
- Currency Ranking
- News Alignment
- `proposes=false`

۲۸. Journal

Journal یکی از Pluginهای سیستم است.

Contract:

```
odin.journal@1
```

مسئولیت:

- **ثبت Signal**
- **Resolve کردن Signalهای باز**
- **محاسبه آمار**
- **کارنامه**

Journal از Strategy مستقل است.

Strategy فقط نتیجه خود را ارائه می‌کند؛ اینکه Signal چه سرنوشتی پیدا کرده، مسئولیت Journal است.

Notification .۲۹

Contract:

```
odin.notify@1
```

در حال حاضر Notification Plugin مسئول ارسال Telegram است.

Strategy نباید مستقیماً Telegram را صدا بزند.

مسیر:

```
Strategy
↓
Judge
↓
Signal
↓
Dispatch
↓
Notify Plugin
```

۳۰. Report

Contract:

```
odin.report@1
```

Report Layer مسئول Presentation است.

منطق تصمیم‌گیری نباید داخل Report قرار بگیرد.

Report می‌تواند اطلاعات زیر را نمایش دهد:

```
Signal
Judge Score
Vetoes
Evidence
Strategy Support
Risk
Journal
```

۳۱. Alerts

Price Alert نیز Plugin مستقل است:

```
odin.alerts.price@1
```

Strategy نباید مسئول Price Alert باشد.

۳۲. Data Layer

Data Providerها Plugin هستند.

Providerهای اصلی:

```
data-yahoo
data-twelvedata
data-auto
data-tradingview
```

Data Contract:

```
odin.data.market@1
```

و TradingView:

```
odin.data.tv@1
```

اصل مهم:

```
No Data
   ≠
Fake Data
```

اگر Provider داده ندارد، وضعیت باید صادقانه unavailable باشد.

۳۳. Fundamental Layer

Fundamental به دو بخش اصلی تقسیم شده:

```
fundamental-calendar
fundamental-news
```

Contractها:

```
odin.fundamental.calendar@1
odin.fundamental.news@1
```

این داده‌ها در:

- **Veto**
- **Evidence**
- **Carry Strategy**

مصرف می‌شوند.

۳۴. Session Layer

Session یک Provider مستقل است:

```
odin.session@1
```

کاربرد:

- **Market Open/Closed**
- **Session Detection**
- **Session Evidence**
- **London Breakout**

۳۵. Runtime Flow کامل

چرخه کامل ODIN در حالت فعلی:

```
START
  │
  ▼
Load Config
  │
  ▼
Build Registry
  │
  ▼
Resolve Dependencies
  │
  ▼
Initialize Plugins
  │
  ▼
Start Plugins
  │
  ▼
collect_market
  │
  ├─── Market Data
  ├─── Technical Analysis
  ├─── Currency Strength
  └─── TradingView
  │
  ▼
journal_pre
  │
  ├─── Resolve Open Signals
  └─── Compute Stats
  │
  ▼
collect_fundamental
  │
  ├─── Calendar
  └─── News
  │
  ▼
compute_vetoes
  │
  └─── Veto Rules
  │
  ▼
judge
  │
  ├─── Setup
  ├─── Veto
  ├─── Evidence
  ├─── Score
```

```
  ├──── Strategy Agreement
  └──── Risk Levels
    │
    ▼
render
    │
    ▼
dispatch_signals
    │
  ├──── Anti Spam
  ├──── Notify
  └──── Journal
    │
    ▼
price_alerts
    │
    ▼
chart_cache
    │
    ▼
dashboard
    │
    ▼
archive_notify
    │
    ▼
END
```

۳۶. Scheduler

Scheduler خارج از Pipeline اصلی قرار دارد.

وظایف زمان‌بندی:

```
event_alerts
briefing
journal_report
```

در Android:

```
OdinService
svcTick
```

مسئول اجرای پس‌زمینه است.

۳۷. Native Boundary

Android Native Layer شامل اجزایی مانند:

```
MainActivity
OdinService
Bridge
BootReceiver
Notif
```

است.

اصل معماری:

```
Business Logic
→ Java/Kotlin منتقل نشود
```

Native Layer باید عمدتاً مسئول:

- **WebView**
- **Lifecycle Android**
- **Background Service**
- **Notification**
- **Bridge**
- **File/Platform Operations**

باشد.

۳۸. استقلال Windows

نسخه Windows در معماری فعلی:

```
Frozen / Oracle
```

است.

توسعه اصلی محصول روی Android انجام می‌شود.

Python همچنان برای:

- **Oracle**
- **Golden Fixtures**
- **Regression**
- **Parity**
- **تحلیل و تست**

اهمیت دارد.

۳۹. قوانین توسعه Plugin جدید

برای افزودن قابلیت جدید، مدل مورد انتظار:

```
1. Contract
2. Plugin Manifest
3. Registry Registration
4. Dependency Definition
5. Lifecycle
6. Pipeline Stage
7. Config Binding
8. Failure Isolation
9. Golden Test
10. Python/JS Parity
```

اما اصل مهم:

> ***قابلیت جدید نباید برای اضافه شدن خود به خود Core را به منطق Feature آلوده کند.***

۴۰. قوانین توسعه آینده

Core --- Rule ۱ را کوچک نگه دار

منطق Business وارد Core نشود.

Plugin --- Rule ۲ها مستقل باشند

هر Feature مرز مشخص داشته باشد.

Contract --- Rule ۳ها پایدار باشند

Breaking Change فقط با Version جدید Contract.

Failure --- Rule ۴ صادقانه باشد

خطا:

```
unavailable
```

نه:

```
fake success
```

Rule ۵ --- رفتار قبلی حفظ شود

Refactor نباید به‌صورت پنهانی رفتار سیستم را تغییر دهد.

Python --- Rule ۶ و JS باید هم‌رفتار بمانند

هر منطق مهم باید قابلیت Golden/Parity Test داشته باشد.

Strategy --- Rule ۷ از Risk جدا بماند

```
Strategy ≠ Risk
```

Strategy --- Rule ۸ از Execution جدا بماند

```
Strategy ≠ Broker
Strategy ≠ MT5
Strategy ≠ Order
```

۴۱. مسیر معماری آینده برای Auto Trading

Auto Trading در Core قرار نخواهد گرفت.

معماری مورد نظر:

```
Market
  ↓
Analysis
  ↓
Strategy
  ↓
Judge
  ↓
Risk Engine
  ↓
Paper Trading
  ↓
Execution Contract
  ↓
Execution Plugin
  ↓
MT5 / Broker / API
```

در این مدل:

```
MT5 = Plugin
Broker = Plugin
Paper Trading = Plugin
```

و Core به هیچ Broker خاصی وابسته نمی‌شود.

۴۲. چرا این معماری برای ODIN مناسب است؟

این معماری باعث می‌شود:

افزودن Strategy

```
New Strategy
  ↓
New Plugin
  ↓
Same Contract
```

بدون بازنویسی Core.

تعویض Data Provider

```
Yahoo
 ↓
TwelveData
```

بدون تغییر تحلیل.

تعویض Notification

```
Telegram
 ↓
Other Provider
```

بدون تغییر Judge.

اضافه شدن Execution

```
Paper Trading
 ↓
MT5
 ↓
Broker API
```

بدون تغییر Strategy.

۴۳. وضعیت فعلی معماری

در نسخه فعلی پروژه، ساختار جدید شامل موارد زیر است:

✓ Core Contracts
✓ Plugin Manifest
✓ Plugin Registry
✓ Dependency Resolver
✓ Lifecycle Manager
✓ Event Bus
✓ Pipeline Runner
✓ Config Bridge
✓ Data Plugins
✓ Analysis Plugins
✓ Fundamental Plugins
✓ Judge Plugins
✓ Veto Plugins
✓ Evidence Plugins
✓ Risk Plugin
✓ Journal Plugin
✓ Notify Plugin
✓ Alerts Plugin
✓ Report Plugins
✓ Strategy Contract
✓ Trend Pullback
✓ London Breakout
✓ Carry Trade
✓ Strategy Registry
✓ Strategy Agreement Gate
✓ Python/JS Strategy Parity
✓ Golden Strategy Fixtures
✓ Plugin Status / Failure Isolation

۴۴. ساختار مفهومی Repository

```
ODIN/
 |
├─── src/
|    ├─── core/
|    |    ├─── contracts.py
|    |    ├─── manifest.py
|    |    ├─── registry.py
|    |    ├─── resolver.py
|    |    ├─── lifecycle.py
|    |    ├─── bus.py
|    |    ├─── pipeline.py
|    |    └─── config_bridge.py
|    |
|    ├─── plugins/
|    |    ├─── data.py
|    |    ├─── analysis.py
|    |    ├─── fundamental.py
|    |    ├─── judge.py
|    |    ├─── strategies.py
|    |    ├─── journal.py
|    |    ├─── notify.py
|    |    ├─── alerts.py
|    |    └─── report.py
|    |
|    ├─── strategies/
|    |    ├─── base.py
|    |    ├─── trend_pullback.py
|    |    ├─── london_breakout.py
|    |    └─── carry.py
|    |
|    ├─── data/
|    ├─── analysis/
|    ├─── fundamental/
|    ├─── judge/
|    ├─── journal/
|    └─── report/
|
├─── android/
|    └─── app/src/main/assets/www/js/
|         ├─── core.js
|         ├─── plugins.js
|         ├─── strategies.js
|         ├─── technical.js
|         ├─── judge.js
|         ├─── data.js
|         └─── ...
```

```
│
├─── tests/
│    ├─── test_plugins.py
│    ├─── test_strategies.py
│    └─── js/
│         ├─── test_strategies_switch.js
│         └─── golden/
│              └─── strategies_golden.json
│
└─── docs/
```

۴۵. تعریف نهایی معماری

معماری فعلی ODIN را می‌توان به‌صورت زیر خلاصه کرد:



اصل نهایی:

> *ODIN یک Core کوچک و پایدار دارد که فقط Contract و Runtime را مدیریت می‌کند؛ تمام قابلیت‌های معاملاتی، تحلیلی، داده‌ای و جانبی به‌صورت Plugin در اطراف آن قرار دارند. Strategyها فقط منطق Setup و توافق را ارائه می‌کنند، Judge تصمیم نهایی تحلیلی را می‌گیرد، Risk مسئول مدیریت ریسک است و Execution در آینده به‌صورت Plugin مستقل اضافه خواهد شد.*

۴۶. مرزهای غیرقابل شکستن

```
Core
  ✗ نباید منطق Strategy داشته باشد
  ✗ نباید منطق Broker داشته باشد
  ✗ نباید Yahoo را بشناسد
  ✗ نباید Telegram را بشناسد

Strategy
  ✗ نباید Order ایجاد کند
  ✗ نباید Broker را صدا بزند
  ✗ نباید SL/TP را مالک تصمیم کند
  ✗ نباید Veto را دور بزند

Judge
  ✗ نباید Broker را صدا بزند

Risk
  ✗ نباید Strategy جدید تعریف کند

Execution
  ✗ نباید منطق تحلیل بازار را مالک شود

UI
  ✗ نباید Business Logic را مالک شود
```

این مرزبندی‌ها پایه اصلی قابل توسعه بودن ODIN هستند.